Subkey Generation in CMAC

36. Write a C program for Caesar cipher, known as the affine Caesar cipher, has the following form: For each plaintext letter p, substitute the ciphertext letter C: C = E([a, b], p) = (ap + b) mod 26 A basic requirement of any encryption algorithm is that it be one-to-one. That is, if p q, then E(k, p) E(k, q). Otherwise, decryption is impossible, because more than one plaintext character maps into the same ciphertext character. The affine Caesar cipher is not one-to-one for all values of a. For example, for a = 2 and b = 3, then E([a, b], 0) = E([a, b], 13) = 3.

Aim:

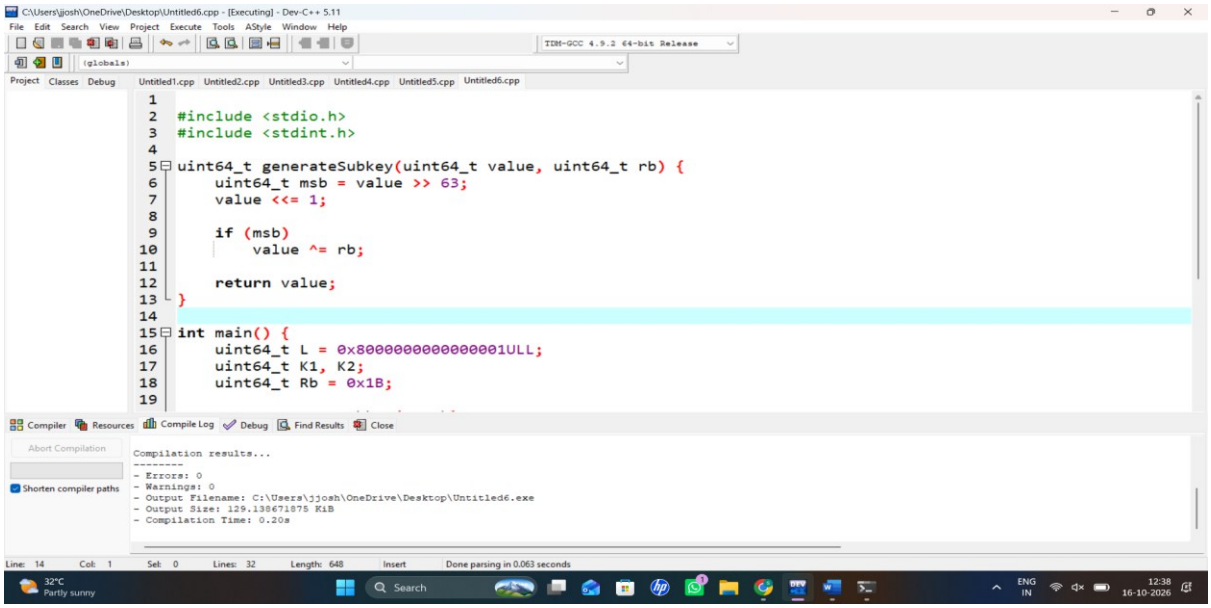
To implement CMAC subkey generation and understand the left-shift and XOR operations used to generate two subkeys.

Algorithm:

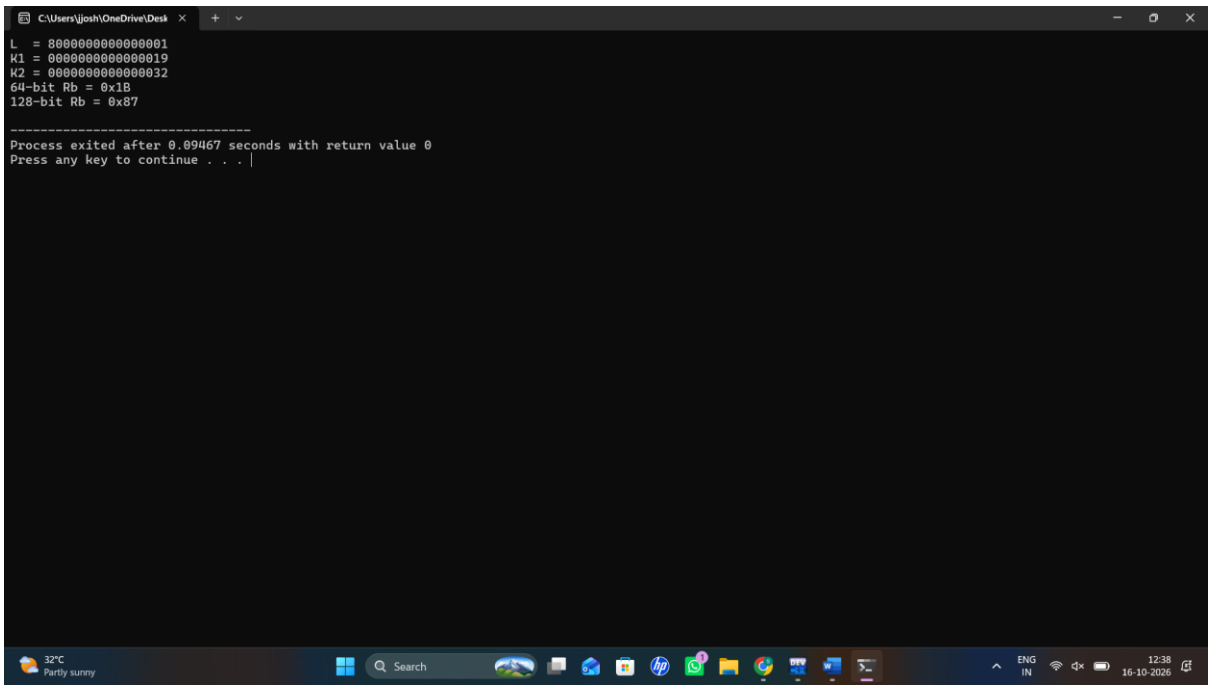
1. Initialize a block of all zeros.
2. Apply the block cipher with secret key \(K\) to obtain \(L=E_K(0)\).
3. Left-shift \(L\) by one bit to generate the first subkey \(K_1\).
4. If the most significant bit of \(L\) is 1, XOR the shifted result with a constant \(R_b\).
5. Left-shift \(K_1\) to generate \(K_2\).
6. If the most significant bit of \(K_1\) is 1, XOR with the same constant.

Program:

This program demonstrates the subkey-generation bit operations using sample block values. It does not implement a real block cipher.



Output:



a. Constants required:

- For a 64-bit block: \(R_b=\texttt{0x1B}\).
- For a 128-bit block: \(R_b=\texttt{0x87}\).

b. How left shift and XOR work:

- Left shift moves every bit one position to the left, effectively multiplying the bit string by 2 modulo the block size.

- If the original most significant bit is 1, the shift discards that bit. XOR with \(R_b\) applies the required reduction for the corresponding finite-field operation.
- The same procedure is applied to \(K_1\) to obtain \(K_2\).

Result:

CMAC subkey generation was demonstrated using left shifts and conditional XOR operations.